

QA TEST PLAN

DemoQA Web Application — demoqa.com

Version 1.0 | Test planning baseline | September 2026

1. Document Control

Item	Details
Document	DemoQA Complete QA Test Plan
Application	DemoQA — Web UI practice/testing application
URL	https://demoqa.com/
Plan Version	1.0
Prepared For	Manual QA, Automation QA and interview/practical testing use
Test Approach	Risk-based functional, negative, UI, accessibility, compatibility, performance smoke and end-to-end testing
Primary Test Artifacts	Test plan, test cases, test data, defect reports, execution report, automation suite
Exit Evidence	Executed critical/high-priority tests, defect disposition, regression result and test summary report

2. Executive Summary

This test plan defines the strategy, scope, environments, responsibilities, test levels, entry/exit criteria, risks and deliverables for validating DemoQA. The current application organizes practice functionality into Elements, Forms, Alerts/Frame/Windows, Widgets, Interactions and Book Store Application areas. The plan therefore combines component-level checks with cross-module and end-to-end scenarios.

The objective is not only to confirm that individual controls work, but also to verify validation, state management, navigation, browser behavior, accessibility fundamentals, error handling and the Book Store user journey.

3. Application Overview

The current DemoQA home structure includes the following major areas:

Area	Key Features in Scope
Elements	Text Box, Check Box, Radio Button, Web Tables, Buttons, Links, Broken Links/Images, Upload and Download, Dynamic Properties
Forms	Practice Form / Student Registration Form
Alerts, Frame & Windows	Browser Windows, Alerts, Frames, Nested Frames, Modal Dialogs
Widgets	Accordion, Auto Complete, Date Picker, Slider, Progress Bar, Tabs, Tool Tips, Menu, Select Menu
Interactions	Sortable, Selectable, Resizable, Droppable, Draggable
Book Store Application	Login, Registration, Book Store, Profile, Book Store API

4. Objectives

- Verify that all in-scope user-facing functions behave according to their intended interaction model.
- Validate positive, negative, boundary and invalid-input behavior.
- Verify data persistence and state transitions where the application supports them.
- Verify browser windows, tabs, JavaScript alerts, frames, nested frames and modal dialogs.
- Verify interactive widgets and mouse/drag-and-drop behaviors.
- Verify the Book Store authentication, search, book details and collection workflows.
- Identify usability, accessibility, compatibility, performance and error-handling risks.

- Provide a regression baseline suitable for future Selenium/Playwright automation.

5. Scope

5.1 In Scope

- Functional testing of all listed DemoQA modules.
- Field validation, required-field behavior, input constraints and form submission.
- CRUD/search behavior for Web Tables.
- File upload/download controls.
- Dynamic UI behavior and state changes.
- Browser tab/window handling and modal/alert interactions.
- Frame and nested-frame switching.
- Widgets, menus, autocomplete and date selection.
- Mouse interactions including drag, drop, resize, sort and select.
- Book Store registration, login, book listing/search/detail, collection and profile flows.
- Cross-browser smoke testing and responsive-layout checks.
- Keyboard accessibility and visible focus checks.
- Basic performance observations and error handling.

5.2 Out of Scope / Conditional

- Production-grade security penetration testing or vulnerability exploitation.
- Load/stress testing at production-scale traffic levels unless a dedicated environment is provided.
- Third-party infrastructure and services outside DemoQA's application boundary.
- Formal WCAG certification; accessibility checks in this plan are baseline QA checks.
- Real-user analytics and business KPIs, unless supplied separately.

6. Test Strategy

Test Type	Purpose	Priority
Smoke	Confirm the application and major modules are reachable and usable.	Critical
Functional	Validate expected behavior of individual features and controls.	Critical
Negative	Validate invalid input, rejected operations and error handling.	High
Integration / Workflow	Validate state/data transitions between related features.	High
End-to-End	Validate complete user journeys, especially Book Store and forms.	Critical
Regression	Re-run stable high-risk tests after changes/fixes.	Critical
UI/Usability	Check layout, labels, visibility, interaction feedback and consistency.	Medium
Accessibility	Keyboard operation, focus visibility and basic operability.	High
Compatibility	Browser and viewport behavior.	High
Performance Smoke	Observe load/usability and obvious delays under normal conditions.	Medium
Automation	Automate stable repeatable regression flows after manual baseline.	High

7. Test Levels

- Component/feature level: individual controls such as text boxes, radio buttons, tables and widgets.
- Integration level: interactions that depend on state or related controls, such as state/city selection and Book Store collection updates.
- System level: complete pages and modules across DemoQA.
- End-to-end level: user journeys from authentication through book search/add/remove, and complete form submission.
- Regression level: repeat critical and high-priority tests after application changes.

8. Functional Coverage Matrix

Module	Coverage	Key Validation
Elements	Text Box; Check Box; Radio Button; Web Tables; Buttons; Links; Broken Links/Images; Upload/Download; Dynamic Properties	Create/update/delete/search records; validation; mouse actions; file handling
Forms	Practice Form	Mandatory/optional fields; email/mobile validation; gender; DOB; subjects; hobbies; image upload; state/city; submission
Alerts/Frames/Windows	Browser Windows; Alerts; Frames; Nested Frames; Modal Dialogs	New tab/window; alert/confirm/prompt; frame switching; modal open/close
Widgets	Accordion; Auto Complete; Date Picker; Slider; Progress Bar; Tabs; Tool Tips; Menu; Select Menu	Expand/collapse; suggestions; dates; slider movement; progress; tabs; hover; selection
Interactions	Sortable; Selectable; Resizable; Droppable; Draggable	Reorder; select; resize constraints; valid/invalid drop; drag movement
Book Store	Login; Register; Book Store; Profile; Book Store API	Authentication; registration validation; search; details; add/remove collection; profile state

9. Test Environment

Category	Baseline
Application URL	https://demoqa.com/
Desktop Browsers	Latest supported/stable Chrome, Microsoft Edge and Firefox
Viewport	Desktop baseline plus tablet/mobile responsive widths
Operating Systems	Windows baseline; add macOS/Linux if the project requires them
Network	Normal broadband; record unusual latency/outage conditions separately
Test Accounts	Dedicated non-production DemoQA test accounts for Book Store flows
Test Data	Synthetic data only; no real credentials or sensitive personal information
Automation Stack (optional)	Selenium or Playwright with Page Object Model and CI execution

10. Test Data Strategy

- Use unique usernames for registration tests to avoid collisions.
- Use deterministic synthetic names, email addresses, phone numbers and addresses.
- Maintain valid and invalid datasets separately.
- Use boundary datasets for mobile numbers, text lengths, numeric fields and date selections where applicable.
- Use known books for search/detail/collection flows; the current Book Store page exposes titles such as Git Pocket Guide and You Don't Know JS.
- Use a small set of reusable files for upload/download validation (valid image, text file and unsupported file type where the control permits testing).
- Reset test data or delete created collection/table records after tests where feasible to prevent cross-test contamination.

11. Entry Criteria

- DemoQA URL is reachable from the test environment.
- Required browsers and test devices/environments are available.
- Test data and dedicated test accounts are prepared.
- Test cases are reviewed and prioritized.
- Any blocking environment issue is documented before execution.
- For automation, stable locators and a clean test-data strategy are available.

12. Exit Criteria

- All Critical tests executed with no unresolved Critical defects.
- At least 95% of planned Critical/High tests executed.
- All failed tests have linked defects or documented environment explanations.
- No open defect classified as release-blocking/critical without explicit acceptance.
- Regression suite passes at the agreed threshold.
- Known limitations and deferred risks are documented in the test summary.

13. Severity and Priority

Severity	Definition	Example
Critical	Core workflow unavailable, data loss/corruption, security-critical or complete blocker.	Cannot log in or access core Book Store workflow for all users.
High	Major feature broken with significant workaround impact.	Cannot add a book to collection or submit valid registration.
Medium	Feature works incorrectly but workaround exists or impact is limited.	Incorrect validation message or non-critical widget behavior.
Low	Cosmetic or minor usability issue.	Minor spacing, alignment or wording issue.

14. Defect Management

- Every failed test should reference a defect ID when the failure is attributable to the application.
- Defects should contain title, environment, preconditions, steps, expected result, actual result, evidence, severity, priority and reproducibility.
- Retest a defect after a fix; then run impacted regression tests.
- If a failure is caused by environment/network/browser instability, record it separately rather than misclassifying it as an application defect.
- Close defects only after successful verification or documented accepted disposition.

15. Risk Register

Risk	Impact	Likelihood	Mitigation
DemoQA content/behavior changes	Test cases or locators may become stale.	Medium	Baseline the application before each cycle; maintain stable locators and review changed modules.
Shared test account/data	Parallel tests may interfere with each other.	Medium	Use unique accounts/data or isolate tests and clean up after execution.
Browser-specific behavior	UI/interaction tests may behave differently.	Medium	Run browser smoke/regression on Chrome, Edge and Firefox.
Drag/drop or dynamic controls are timing-sensitive	Automation may be flaky.	Medium	Use explicit waits and state-based assertions; avoid arbitrary sleeps.
External network instability	False failures.	Medium	Record network conditions and rerun failed tests in a stable environment.
Accessibility coverage is incomplete	Potential accessibility defects may escape.	Medium	Use keyboard checks and supplement with dedicated accessibility tooling when required.
Test data contamination	Later tests may inherit changed state.	High	Use cleanup hooks, unique data and isolated test accounts.

16. Smoke Test Suite

- Home page loads and primary modules are visible.
- Text Box accepts valid data and submits.

- Web Tables loads and supports add/search/edit/delete.
- Practice Form accepts valid mandatory data and submits.
- Browser Windows opens a new tab/window.
- Alert appears and can be accepted.
- Frames can be switched into and out of.
- Accordion/Auto Complete/Date Picker loads and responds.
- Drag-and-drop or sortable interaction works.
- Book Store login page loads; authenticated user can access Book Store/Profile.
- Book search returns a known book and book details open.

17. Regression Strategy

Regression should be risk-based. After any change, execute the smoke suite first, then all tests directly affected by the change, followed by Critical/High end-to-end and integration flows. The previously created DemoQA Excel suite can serve as the execution baseline.

Change Area	Minimum Regression
Elements/UI controls	Elements smoke + affected control tests + navigation
Practice Form	All form validation/submission tests + responsive/accessibility checks
Alerts/Frames/Windows	All window/alert/frame tests
Widgets/Interactions	Affected widget/interaction tests + browser compatibility
Book Store/authentication	Login/register + search/detail + add/remove collection + profile + E2E
Shared CSS/navigation/layout	Cross-module smoke + responsive + accessibility + browser smoke

18. Automation Strategy

- Automate stable, repeatable Critical/High functional and regression scenarios first.
- Use Page Object Model or a comparable abstraction for maintainability.
- Use explicit waits based on application state; avoid fixed sleeps except when technically unavoidable.
- Keep test data isolated and generate unique values where required.
- Tag tests by smoke, regression, module and priority.
- Capture screenshots, DOM/page source and logs on failure.
- Run a fast smoke suite on every build/change and a broader regression suite on scheduled CI runs.
- Keep UI tests independent where possible; reset application state between tests.

19. Accessibility Test Approach

- Navigate interactive controls using Tab and Shift+Tab.
- Verify a visible focus indicator.
- Verify buttons, links and form controls have understandable labels.
- Verify keyboard-operable controls such as accordions, menus, tabs and dialogs.
- Check that validation errors are visible and understandable.
- Supplement manual checks with automated accessibility tooling such as axe where the project permits.
- Record accessibility findings separately from general visual defects when useful.

20. Compatibility Test Approach

Dimension	Coverage
Browsers	Chrome, Edge, Firefox; add Safari when required by project/user base.
Viewport	Desktop, tablet and mobile widths.
Input	Mouse and keyboard; touch where applicable.

Window behavior	Tabs, separate windows, dialogs and responsive layout.
Locale/data	Dates, text, numeric fields and validation using representative input.

21. Performance Test Approach

- This plan provides performance smoke coverage, not full load testing.
- Record obvious slow loads, delayed controls, stalled progress bars and unresponsive interactions.
- For repeatable metrics, capture browser performance timings and compare against a baseline.
- If formal performance targets are required, define measurable thresholds and run dedicated load/stress testing in an appropriate environment.

22. Traceability

Recommended traceability chain:

1. Requirement / feature
2. Test scenario
3. Detailed test case
4. Execution result
5. Defect, if failed
6. Retest result
7. Regression result

The Test Case ID from the Excel suite should be the primary key used in execution and defect references.

23. Deliverables

Deliverable	Owner	When
QA Test Plan	QA Lead	Before execution
Test Case Suite	QA	Before execution
Test Data	QA	Before execution and maintained during cycle
Automation Suite	Automation QA	Incrementally during/after baseline manual validation
Defect Reports	QA	As failures are discovered
Daily/Periodic Execution Report	QA Lead	During execution
Regression Report	QA	After fixes/change window
Test Summary Report	QA Lead	At test-cycle closure

24. Entry-to-Exit Execution Flow

8. Review scope and application changes.
9. Prepare environment, accounts and test data.
10. Execute smoke tests.
11. Execute Critical/High functional tests.
12. Execute negative, UI, accessibility and compatibility checks.
13. Execute integration and end-to-end workflows.
14. Log, triage and retest defects.
15. Run regression after fixes.
16. Evaluate exit criteria.
17. Publish test summary and residual-risk statement.

25. Recommended QA Metrics

Metric	Formula / Definition
Execution %	Executed test cases / planned test cases × 100
Pass %	Passed / executed × 100
Fail %	Failed / executed × 100
Blocked %	Blocked / planned × 100
Defect density	Valid defects / executed test cases
Critical defect count	Open Critical defects at reporting time
Regression pass rate	Passed regression cases / executed regression cases × 100
Automation coverage	Automated in-scope cases / total in-scope cases × 100

26. Current Application Evidence / Reference Points

The plan was aligned to the current DemoQA structure observed during planning. The home page lists the six main areas and their features; the Practice Form exposes name, email, gender, mobile, DOB, subjects, hobbies, picture, address and state/city inputs; the Book Store exposes search, login, book listing and book details; and dedicated pages exist for alerts, frames, windows, modals and interactive widgets.

27. Final Acceptance Recommendation

DemoQA should be considered QA-complete for a test cycle when the agreed Critical/High scope has been executed, all blocking defects are resolved or formally accepted, the regression baseline is green at the agreed threshold, and residual risks are documented. Because DemoQA is a practice site rather than a conventional production product, the exact release-go/no-go threshold should be adapted to the learning, interview, automation-framework or demonstration objective.

Appendix A — Suggested Test Case Fields

Field	Purpose
Test Case ID	Unique identifier such as TC-001.
Module / Feature	Functional area under test.
Scenario	Short statement of what is being validated.
Preconditions	Required state before execution.
Test Steps	Ordered user actions.
Test Data	Values/files/accounts used.
Expected Result	Observable acceptance behavior.
Actual Result	Observed result during execution.
Status	Not Run / Pass / Fail / Blocked / N/A.
Priority	Critical / High / Medium / Low.
Severity	Impact classification for defects.
Defect ID	Linked defect when applicable.
Evidence	Screenshot/log/video reference.
Automation Status	Manual / Automated / Candidate / Not Suitable.

Appendix B — Core E2E Scenarios

- Register → Login → Search Book → Open Book Details → Add to Collection → Profile → Remove Book.
- Practice Form → Populate all relevant fields → Upload picture → Submit → Verify submission data.
- Web Tables → Add unique record → Search → Edit → Search updated value → Delete → Verify removal.
- Browser Windows → Open New Tab/Window → Verify content → Return to original context.
- Alerts → Simple Alert → Confirm Accept/Dismiss → Prompt Input → Verify resulting messages.
- Frames → Enter parent frame → Enter nested child frame → Verify content → Return to parent/default content.
- Interactions → Sort/drag/drop/resize/select → Verify final UI state.